Percona Operator for PostgreSQL

DBA Tasks Automation Scripts Runbook

Production-Ready Shell Scripts for Kubernetes PostgreSQL Administration on Percona Operator v2.x

Platform: AlmaLinux 9 / Kubernetes Cluster / Percona Operator v2.x

PostgreSQL Versions: 14.x — 17.x

Automation: Kubernetes CronJobs + Bastion Scripts

Table of Contents

PART I — FOUNDATION: CORE WRAPPER LIBRARY

Core Wrapper Library

Every automation script sources this shared library. It handles primary pod discovery, SQL execution, logging, error handling, and Patroni interaction. Store this as a ConfigMap and mount it into all DBA automation pods.

pg-dba-lib.sh

Core library — source this from every DBA automation script

```
#!/bin/bash
# pg-dba-lib.sh — Core DBA library for Percona Operator PostgreSQL
# Source this file: . /scripts/pg-dba-lib.sh
set -euo pipefail

# ■■ Configuration (override via environment) ■■■■■■■■■■■■■■■■■■
NAMESPACE="${PG_NAMESPACE:-pg-production}"
CLUSTER="${PG_CLUSTER:-my-cluster}"
CONTAINER="${PG_CONTAINER:-database}"
PG_SUPERUSER="${PG_SUPERUSER:-postgres}"
LOG_DIR="${DBA_LOG_DIR:-/tmp/dba-logs}"
LOG_FILE="$LOG_DIR/dba-$(date +%Y%m%d).log"
ALERT_LOG="$LOG_DIR/alerts-$(date +%Y%m%d).log"
EXIT_CODE=0

mkdir -p "$LOG_DIR"

# ■■ Logging ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
log() { echo "[$(date +'%Y-%m-%d %H:%M:%S')] [INFO] $*" | tee -a "$LOG_FILE"; }
warn() { echo "[$(date +'%Y-%m-%d %H:%M:%S')] [WARN] $*" | tee -a "$LOG_FILE" "$ALERT_LOG"; }
error() { echo "[$(date +'%Y-%m-%d %H:%M:%S')] [ERROR] $*" | tee -a "$LOG_FILE" "$ALERT_LOG";
EXIT_CODE=1; }
sep() { log "■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■"; }

# ■■ Pod Discovery ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
get_primary() {
kubectl get pod -n "$NAMESPACE" \
-l postgres-operator.crunchydata.com/role=master,\
postgres-operator.crunchydata.com/cluster="$CLUSTER" \
-o jsonpath='{.items[0].metadata.name}' 2>/dev/null
}

get_replicas() {
kubectl get pod -n "$NAMESPACE" \
-l postgres-operator.crunchydata.com/role=replica,\
postgres-operator.crunchydata.com/cluster="$CLUSTER" \
-o jsonpath='{.items[*].metadata.name}' 2>/dev/null
}

PRIMARY=$(get_primary)
```

```
if [[ -z "$PRIMARY" ]]; then
error "Cannot find primary pod for cluster $CLUSTER"
exit 1
fi
log "Primary pod: $PRIMARY"

# ■■ SQL Execution ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
run_sql() {
local sql="$1"
local db="${2:-postgres}"
local result
result=$(kubectl exec -n "$NAMESPACE" "$PRIMARY" -c "$CONTAINER" -- \
psql -U "$PG_SUPERUSER" -d "$db" -At -c "$sql" 2>&1) || {
error "SQL failed on $db: ${sql:0:80}..."
error "Output: $result"
return 1
}
echo "$result"
}

run_sql_verbose() {
local sql="$1"
local db="${2:-postgres}"
kubectl exec -n "$NAMESPACE" "$PRIMARY" -c "$CONTAINER" -- \
psql -U "$PG_SUPERUSER" -d "$db" -c "$sql" 2>&1 | tee -a "$LOG_FILE"
}

run_sql_file() {
local file="$1"
local db="${2:-postgres}"
log "Executing SQL file: $file on $db"
kubectl exec -i -n "$NAMESPACE" "$PRIMARY" -c "$CONTAINER" -- \
psql -U "$PG_SUPERUSER" -d "$db" < "$file" 2>&1 | tee -a "$LOG_FILE"
}

# ■■ Patroni ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
run_patronictl() {
kubectl exec -n "$NAMESPACE" "$PRIMARY" -c "$CONTAINER" -- \
patronictl "$@" 2>&1
}

# ■■ Database Listing ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
list_databases() {
run_sql "SELECT datname FROM pg_database
WHERE datistemplate = false
AND datname NOT IN ('postgres')
ORDER BY datname;"
}

# ■■ Completion Handler ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
finish() {
if [[ $EXIT_CODE -eq 0 ]]; then
log "Script completed successfully"
else
warn "Script completed with errors — check $ALERT_LOG"
```

```
fi
exit $EXIT_CODE
}
trap finish EXIT

log "DBA library loaded for cluster=$CLUSTER namespace=$NAMESPACE"
```

Every script in this runbook begins with `source /scripts/pg-dba-lib.sh` to inherit all helper functions, logging, and pod discovery.

PART II — USER AND ROLE MANAGEMENT AUTOMATION

`manage-users.sh`

Create, modify, and audit PostgreSQL users and roles

```
#!/bin/bash
# manage-users.sh — User and role lifecycle management
source /scripts/pg-dba-lib.sh

ACTION="${1:-audit}" # create|delete|audit|rotate-password
USERNAME="${2:-}"
TARGET_DB="${3:-}"

sep
log "User Management: action=$ACTION user=$USERNAME db=$TARGET_DB"
sep

case "$ACTION" in

create)
[[ -z "$USERNAME" || -z "$TARGET_DB" ]] && {
error "Usage: $0 create <username> <database>"; exit 1; }

# Generate a secure random password
PASSWORD=$(openssl rand -base64 24 | tr -dc 'A-Za-z0-9' | head -c 20)

log "Creating role: $USERNAME"
run_sql "DO \$\$
BEGIN
IF NOT EXISTS (SELECT FROM pg_roles WHERE rolname = '$USERNAME') THEN
CREATE ROLE $USERNAME LOGIN PASSWORD '$PASSWORD'
NOSUPERUSER NOCREATEDB NOCREATEROLE;
RAISE NOTICE 'Role % created', '$USERNAME';
ELSE
RAISE NOTICE 'Role % already exists', '$USERNAME';
END IF;
END
\$\$;"

log "Granting CONNECT on $TARGET_DB"
run_sql "GRANT CONNECT ON DATABASE $TARGET_DB TO $USERNAME;"

log "Granting schema privileges"
run_sql "GRANT USAGE ON SCHEMA public TO $USERNAME;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO $USERNAME;
ALTER DEFAULT PRIVILEGES IN SCHEMA public
GRANT SELECT ON TABLES TO $USERNAME;" "$TARGET_DB"

# Store password in Kubernetes secret
kubectl create secret generic "pguser-$USERNAME" \
-n "$NAMESPACE" \
--from-literal=username="$USERNAME" \
--from-literal=password="$PASSWORD" \
```

```
--from-literal=database="$TARGET_DB" \
--dry-run=client -o yaml | kubectl apply -f -

log "User $USERNAME created. Credentials stored in secret pguser-$USERNAME"
;;

delete)
[[ -z "$USERNAME" ]] && { error "Usage: $0 delete <username>"; exit 1; }
log "Deleting role: $USERNAME"

# Reassign and drop in all databases
for DB in $(list_databases); do
log " Processing database: $DB"
run_sql "REASSIGN OWNED BY $USERNAME TO postgres;" "$DB" 2>/dev/null || true
run_sql "DROP OWNED BY $USERNAME;" "$DB" 2>/dev/null || true
done

run_sql "DROP ROLE IF EXISTS $USERNAME;"
kubectl delete secret "pguser-$USERNAME" -n "$NAMESPACE" 2>/dev/null || true
log "Role $USERNAME deleted"
;;

rotate-password)
[[ -z "$USERNAME" ]] && { error "Usage: $0 rotate-password <username>"; exit 1; }
NEW_PASS=$(openssl rand -base64 24 | tr -dc 'A-Za-z0-9' | head -c 20)
run_sql "ALTER ROLE $USERNAME PASSWORD '$NEW_PASS';"
# Update the K8s secret
kubectl create secret generic "pguser-$USERNAME" \
-n "$NAMESPACE" \
--from-literal=username="$USERNAME" \
--from-literal=password="$NEW_PASS" \
--dry-run=client -o yaml | kubectl apply -f -
log "Password rotated for $USERNAME"
;;

audit)
log "=== User Audit Report ==="
run_sql_verbose "
SELECT rolname, rolsuper, rolcreatedb, rolcreaterole,
rolcanlogin, rolconnlimit, rolvaliduntil
FROM pg_roles
WHERE rolname NOT LIKE 'pg_%'
AND rolname NOT IN ('postgres')
ORDER BY rolname;
"

log "=== Users with No Login Activity (30+ days) ==="
run_sql_verbose "
SELECT r.rolname,
CASE WHEN s.last_login IS NULL THEN 'Never'
ELSE s.last_login::text END AS last_login
FROM pg_roles r
LEFT JOIN (
SELECT usename, max(backend_start) AS last_login
FROM pg_stat_activity
```

```
GROUP BY usename
) s ON r.rolname = s.usename
WHERE r.rolcanlogin = true
AND r.rolname NOT LIKE 'pg_%';
"

log "=== Role Memberships ==="
run_sql_verbose "
SELECT r.rolname AS role,
m.rolname AS member
FROM pg_auth_members am
JOIN pg_roles r ON am.roleid = r.oid
JOIN pg_roles m ON am.member = m.oid
ORDER BY r.rolname, m.rolname;
"
;;

*)
error "Unknown action: $ACTION"
error "Usage: $0 {create|delete|rotate-password|audit} [username] [database]"
;;
esac
```

PART III — DATABASE LIFECYCLE AUTOMATION

manage-databases.sh

Create, drop, clone, and audit databases

```
#!/bin/bash
# manage-databases.sh — Database lifecycle management
source /scripts/pg-dba-lib.sh

ACTION="${1:-list}"
DBNAME="${2:-}"
OWNER="${3:-postgres}"

sep
log "Database Management: action=$ACTION db=$DBNAME owner=$OWNER"
sep

case "$ACTION" in

create)
[[ -z "$DBNAME" ]] && { error "Usage: $0 create <dbname> [owner]"; exit 1; }

EXISTS=$(run_sql "SELECT 1 FROM pg_database WHERE datname='$DBNAME';")
if [[ "$EXISTS" == "1" ]]; then
warn "Database $DBNAME already exists"
else
log "Creating database: $DBNAME (owner: $OWNER)"
run_sql "CREATE DATABASE $DBNAME OWNER $OWNER
ENCODING 'UTF8'
LC_COLLATE 'en_US.UTF-8'
LC_CTYPE 'en_US.UTF-8';"

# Create standard schemas
log "Creating standard schemas"
run_sql "CREATE SCHEMA IF NOT EXISTS app;
CREATE SCHEMA IF NOT EXISTS audit;
ALTER SCHEMA app OWNER TO $OWNER;
ALTER SCHEMA audit OWNER TO $OWNER;" "$DBNAME"

# Install standard extensions
log "Installing standard extensions"
run_sql "CREATE EXTENSION IF NOT EXISTS pg_stat_statements;
CREATE EXTENSION IF NOT EXISTS pgcrypto;
CREATE EXTENSION IF NOT EXISTS pg_trgm;" "$DBNAME"

log "Database $DBNAME created successfully"
fi
;;

drop)
[[ -z "$DBNAME" ]] && { error "Usage: $0 drop <dbname>"; exit 1; }
log "WARNING: Dropping database $DBNAME"
```

```
# Terminate active connections
CONNS=$(run_sql "SELECT count(*) FROM pg_stat_activity
WHERE datname='$DBNAME' AND pid != pg_backend_pid();")
if [[ "$CONNS" -gt 0 ]]; then
log "Terminating $CONNS active connections"
run_sql "SELECT pg_terminate_backend(pid)
FROM pg_stat_activity
WHERE datname='$DBNAME' AND pid != pg_backend_pid();"
sleep 2
fi

run_sql "DROP DATABASE IF EXISTS $DBNAME;"
log "Database $DBNAME dropped"
;;

clone)
TEMPLATE="${2:-}"
DBNAME="${3:-}"
[[ -z "$TEMPLATE" || -z "$DBNAME" ]] && {
error "Usage: $0 clone <template_db> <new_db>"; exit 1; }

# Terminate connections to template
run_sql "SELECT pg_terminate_backend(pid) FROM pg_stat_activity
WHERE datname='$TEMPLATE' AND pid != pg_backend_pid();" || true
sleep 1

log "Cloning $TEMPLATE -> $DBNAME"
run_sql "CREATE DATABASE $DBNAME TEMPLATE $TEMPLATE OWNER $OWNER;"
log "Database $DBNAME cloned from $TEMPLATE"
;;

list)
log "=== Database Inventory ==="
run_sql_verbose "
SELECT d.datname AS database,
pg_size_pretty(pg_database_size(d.datname)) AS size,
r.rolname AS owner,
numbackends AS active_conns,
d.datcollate AS collation
FROM pg_database d
JOIN pg_roles r ON d.datdba = r.oid
JOIN pg_stat_database sd ON d.oid = sd.datid
WHERE d.datistemplate = false
ORDER BY pg_database_size(d.datname) DESC;
"
;;

*)
error "Usage: $0 {create|drop|clone|list} [args...]"
;;
esac
```

PART IV — SCHEMA AND EXTENSION MANAGEMENT

manage-schemas.sh

Create schemas and manage extensions across databases

```
#!/bin/bash
# manage-schemas.sh — Schema and extension management
source /scripts/pg-dba-lib.sh

ACTION="${1:-audit}"
DBNAME="${2:-}"

case "$ACTION" in

create-schema)
SCHEMA="${3:-}"
OWNER="${4:-postgres}"
[[ -z "$DBNAME" || -z "$SCHEMA" ]] && {
error "Usage: $0 create-schema <db> <schema> [owner]"; exit 1; }

run_sql "CREATE SCHEMA IF NOT EXISTS $SCHEMA AUTHORIZATION $OWNER;" "$DBNAME"
run_sql "ALTER DEFAULT PRIVILEGES FOR ROLE $OWNER IN SCHEMA $SCHEMA
GRANT SELECT ON TABLES TO PUBLIC;" "$DBNAME"
log "Schema $SCHEMA created in $DBNAME (owner: $OWNER)"
;;

install-extensions)
[[ -z "$DBNAME" ]] && { error "Usage: $0 install-extensions <db>"; exit 1; }

EXTENSIONS="pg_stat_statements pgcrypto pg_trgm"
for EXT in $EXTENSIONS; do
log "Installing $EXT in $DBNAME"
run_sql "CREATE EXTENSION IF NOT EXISTS $EXT;" "$DBNAME" || true
done
;;

update-extensions)
log "=== Updating Extensions Across All Databases ==="
for DB in $(list_databases); do
log "Checking extensions in $DB"
OUTDATED=$(run_sql "
SELECT name || ':' || installed_version || '->' || default_version
FROM pg_available_extensions
WHERE installed_version IS NOT NULL
AND installed_version != default_version;" "$DB")

if [[ -n "$OUTDATED" ]]; then
log " Outdated extensions: $OUTDATED"
for EXT_NAME in $(run_sql "
SELECT name FROM pg_available_extensions
WHERE installed_version IS NOT NULL
AND installed_version != default_version;" "$DB"); do
log " Updating $EXT_NAME in $DB"
```

```
run_sql "ALTER EXTENSION $EXT_NAME UPDATE;" "$DB" || true
done
else
log " All extensions up to date"
fi
done
;;

audit)
log "=== Schema and Extension Audit ==="
for DB in $(list_databases); do
log "--- Database: $DB ---"
run_sql_verbose "
SELECT schema_name, schema_owner
FROM information_schema.schemata
WHERE schema_name NOT IN ('pg_catalog','information_schema',
'pg_toast','crunchy_pgbouncer')
ORDER BY schema_name;" "$DB"

run_sql_verbose "
SELECT extname, extversion, n.nspname AS schema
FROM pg_extension e
JOIN pg_namespace n ON e.extnamespace = n.oid
ORDER BY extname;" "$DB"
done
;;
esac
```

PART V — BACKUP AUTOMATION AND VERIFICATION

Scheduled backups are handled by the CR's `spec.backups.schedules` section. These scripts handle supplementary tasks: on-demand backups, backup verification, and PITR readiness checks.

`backup-ops.sh`

On-demand backups, verification, and PITR readiness checks

```
#!/bin/bash
# backup-ops.sh — Backup operations and verification
source /scripts/pg-dba-lib.sh

ACTION="${1:-status}"

case "$ACTION" in

trigger)
# Trigger an on-demand backup via PerconaPGBackup CR
BACKUP_NAME="adhoc-$(date +%Y%m%d-%H%M%S)"
REPO="${2:-repo1}"
BACKUP_TYPE="${3:-full}"

log "Triggering $BACKUP_TYPE backup: $BACKUP_NAME (repo: $REPO)"
cat <<EOF | kubectl apply -f -
apiVersion: pgv2.percona.com/v2
kind: PerconaPGBackup
metadata:
name: $BACKUP_NAME
namespace: $NAMESPACE
spec:
pgCluster: $CLUSTER
repoName: $REPO
options:
- --type=$BACKUP_TYPE
EOF

log "Backup $BACKUP_NAME triggered. Monitoring..."
for i in $(seq 1 60); do
STATE=$(kubectl get perconapgbackup "$BACKUP_NAME" -n "$NAMESPACE" \
-o jsonpath='{.status.state}' 2>/dev/null || echo "pending")
if [[ "$STATE" == "Succeeded" ]]; then
log "Backup $BACKUP_NAME completed successfully"
break
elif [[ "$STATE" == "Failed" ]]; then
error "Backup $BACKUP_NAME FAILED"
break
fi
sleep 10
done
;;

status)
log "=== Backup Status ==="
```

```
kubectl get perconapgbackup -n "$NAMESPACE" \
--sort-by=.metadata.creationTimestamp | tail -10 | tee -a "$LOG_FILE"

log "=== pgBackRest Info ==="
kubectl exec -n "$NAMESPACE" "$PRIMARY" -c "$CONTAINER" -- \
pgbackrest info 2>&1 | tee -a "$LOG_FILE"
;;

verify-pitr)
log "=== PITR Readiness Check ==="
# Check WAL archiving status
WAL_STATUS=$(run_sql "
SELECT last_archived_wal,
last_archived_time,
last_failed_wal,
last_failed_time
FROM pg_stat_archiver;")
log "Archiver status: $WAL_STATUS"

# Check replication slots
run_sql_verbose "
SELECT slot_name, slot_type, active,
pg_size_pretty(
pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)
) AS slot_lag
FROM pg_replication_slots;"

# Check latest restorable time
LATEST_RESTORE=$(kubectl get perconapgcluster "$CLUSTER" -n "$NAMESPACE" \
-o jsonpath='{.status.backups.pgbackrest.repos[0].latestRestorableTime}' \
2>/dev/null)
log "Latest restorable time: $LATEST_RESTORE"

# Check WAL archive lag
ARCHIVE_LAG=$(run_sql "
SELECT EXTRACT(EPOCH FROM (now() - last_archived_time))::int
FROM pg_stat_archiver;")
if [[ "$ARCHIVE_LAG" -gt 300 ]]; then
warn "WAL archive lag is ${ARCHIVE_LAG}s (threshold: 300s)"
else
log "WAL archive lag: ${ARCHIVE_LAG}s (OK)"
fi
;;

cleanup-old)
# Clean up completed PerconaPGBackup CRs older than 7 days
log "Cleaning up old backup CRs"
CUTOFF=$(date -d "7 days ago" +%Y-%m-%dT%H:%M:%SZ 2>/dev/null ||
date -v-7d +%Y-%m-%dT%H:%M:%SZ)
kubectl get perconapgbackup -n "$NAMESPACE" \
-o json | jq -r \
".items[] | select(.status.state==\"Succeeded\") |
select(.metadata.creationTimestamp < \"$CUTOFF\") |
.metadata.name" | while read -r NAME; do
log " Deleting old backup CR: $NAME"
```

```
kubectl delete perconapgbackup "$NAME" -n "$NAMESPACE"
done
;;
esac
```

PART VI — DAILY MAINTENANCE: VACUUM, ANALYZE, REINDEX

daily-maintenance.sh

Targeted VACUUM, ANALYZE, and REINDEX across all databases

```
#!/bin/bash
# daily-maintenance.sh — Daily PostgreSQL maintenance
source /scripts/pg-dba-lib.sh

BLOAT_THRESHOLD="${BLOAT_THRESHOLD:-20}" # percent dead tuples
ANALYZE_STALENESS="${ANALYZE_STALENESS:-24}" # hours since last analyze

sep
log "=== Daily Maintenance Starting ==="
sep

for DB in $(list_databases); do
log "Processing database: $DB"

# ■■ Tables needing VACUUM (dead tuple ratio > threshold) ■■
log " Checking for bloated tables (threshold: ${BLOAT_THRESHOLD}%)"
BLOATED=$(run_sql "
SELECT schemaname || '.' || relname
FROM pg_stat_user_tables
WHERE n_live_tup > 1000
AND n_dead_tup > 0
AND round(100.0 * n_dead_tup / NULLIF(n_live_tup + n_dead_tup, 0), 2)
> $BLOAT_THRESHOLD
ORDER BY n_dead_tup DESC
LIMIT 20;" "$DB")

if [[ -n "$BLOATED" ]]; then
for TABLE in $BLOATED; do
log " VACUUM ANALYZE: $TABLE"
run_sql "VACUUM (VERBOSE, ANALYZE) $TABLE;" "$DB" || true
done
else
log " No tables exceed bloat threshold"
fi

# ■■ Tables needing ANALYZE (stale statistics) ■■■■■■■■■■■■■
log " Checking for stale statistics (threshold: ${ANALYZE_STALENESS}h)"
STALE=$(run_sql "
SELECT schemaname || '.' || relname
FROM pg_stat_user_tables
WHERE n_live_tup > 500
AND (last_analyze IS NULL AND last_autoanalyze IS NULL
OR GREATEST(
COALESCE(last_analyze, '1970-01-01'),
COALESCE(last_autoanalyze, '1970-01-01')
) < now() - interval '$ANALYZE_STALENESS hours')
ORDER BY n_live_tup DESC
LIMIT 30;" "$DB")
```

```
if [[ -n "$STALE" ]]; then
for TABLE in $STALE; do
log " ANALYZE: $TABLE"
run_sql "ANALYZE $TABLE;" "$DB" || true
done
else
log " All table statistics are current"
fi

# ■■ Invalid indexes ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
log " Checking for invalid indexes"
INVALID_IDX=$(run_sql "
SELECT schemaname || '.' || indexrelname
FROM pg_stat_user_indexes ui
JOIN pg_index i ON ui.indexrelid = i.indexrelid
WHERE NOT i.indisvalid;" "$DB")

if [[ -n "$INVALID_IDX" ]]; then
for IDX in $INVALID_IDX; do
warn " REINDEX (invalid): $IDX"
run_sql "REINDEX INDEX CONCURRENTLY $IDX;" "$DB" || true
done
else
log " No invalid indexes found"
fi

done

sep
log "=== Daily Maintenance Complete ==="
```

PART VII — PERFORMANCE MONITORING AND QUERY MANAGEMENT

`performance-monitor.sh`

Top queries, long-running query detection, and auto-kill

```
#!/bin/bash
# performance-monitor.sh — Performance monitoring and query management
source /scripts/pg-dba-lib.sh

ACTION="${1:-report}"
KILL_THRESHOLD="${KILL_THRESHOLD:-1800}" # seconds (30 min default)
IDLE_TXN_THRESHOLD="${IDLE_TXN_THRESHOLD:-600}" # seconds (10 min)

case "$ACTION" in

report)
sep
log "=== Performance Report ==="
sep

log "--- Top 10 Queries by Total Time ---"
run_sql_verbose "
SELECT queryid,
LEFT(query, 60) AS query_preview,
calls,
round(total_exec_time::numeric / 1000, 2) AS total_sec,
round(mean_exec_time::numeric, 2) AS avg_ms,
rows
FROM pg_stat_statements
WHERE userid != 10
ORDER BY total_exec_time DESC
LIMIT 10;"

log "--- Active Connections by State ---"
run_sql_verbose "
SELECT state, count(*),
max(EXTRACT(EPOCH FROM (now() - state_change)))::int AS max_age_sec
FROM pg_stat_activity
WHERE pid != pg_backend_pid()
GROUP BY state
ORDER BY count DESC;"

log "--- Connection Counts by User ---"
run_sql_verbose "
SELECT usename, datname, count(*),
count(*) FILTER (WHERE state = 'active') AS active,
count(*) FILTER (WHERE state = 'idle') AS idle,
count(*) FILTER (WHERE state = 'idle in transaction') AS idle_txn
FROM pg_stat_activity
WHERE pid != pg_backend_pid()
GROUP BY usename, datname
ORDER BY count DESC;"
```

```
log "--- Cache Hit Ratios ---"
run_sql_verbose "
SELECT datname,
round(100.0 * blks_hit /
NULLIF(blks_hit + blks_read, 0), 2) AS cache_hit_pct
FROM pg_stat_database
WHERE datname NOT LIKE 'template%'
ORDER BY datname;"
;;

kill-long)
log "=== Killing Long-Running Queries (> ${KILL_THRESHOLD}s) ==="
VICTIMS=$(run_sql "
SELECT pid, usename, datname,
EXTRACT(EPOCH FROM (now() - query_start))::int AS duration_sec,
LEFT(query, 60) AS query
FROM pg_stat_activity
WHERE state = 'active'
AND query NOT LIKE '%pg_stat_activity%'
AND EXTRACT(EPOCH FROM (now() - query_start)) > $KILL_THRESHOLD
AND pid != pg_backend_pid();")

if [[ -n "$VICTIMS" ]]; then
log "$VICTIMS"
PIDS=$(run_sql "
SELECT pid FROM pg_stat_activity
WHERE state = 'active'
AND query NOT LIKE '%pg_stat_activity%'
AND EXTRACT(EPOCH FROM (now() - query_start)) > $KILL_THRESHOLD
AND pid != pg_backend_pid();")
for PID in $PIDS; do
log " Cancelling PID $PID"
run_sql "SELECT pg_cancel_backend($PID);" || true
sleep 2
# Force terminate if still running
STILL=$(run_sql "SELECT 1 FROM pg_stat_activity WHERE pid=$PID AND state='active';")
if [[ "$STILL" == "1" ]]; then
warn " Force terminating PID $PID"
run_sql "SELECT pg_terminate_backend($PID);" || true
fi
done
else
log "No queries exceed threshold"
fi
;;

kill-idle-txn)
log "=== Killing Idle-in-Transaction (> ${IDLE_TXN_THRESHOLD}s) ==="
KILLED=$(run_sql "
SELECT pg_terminate_backend(pid),
usename, datname,
EXTRACT(EPOCH FROM (now()-state_change))::int AS idle_sec
FROM pg_stat_activity
WHERE state = 'idle in transaction'
AND EXTRACT(EPOCH FROM (now()-state_change)) > $IDLE_TXN_THRESHOLD
```

```
AND pid != pg_backend_pid();")
log "Terminated idle transactions: $KILLED"
;;

reset-stats)
log "Resetting pg_stat_statements"
run_sql "SELECT pg_stat_statements_reset();"
log "Stats reset complete"
;;
esac
```

PART VIII — BLOAT DETECTION AND REMEDIATION

bloat-report.sh

Table and index bloat detection with remediation recommendations

```
#!/bin/bash
# bloat-report.sh — Bloat detection and remediation
source /scripts/pg-dba-lib.sh

sep
log "=== Bloat Detection Report ==="
sep

for DB in $(list_databases); do
log "\n--- Database: $DB ---"

log " Top 15 tables by dead tuple ratio:"
run_sql_verbose "
SELECT schemaname || '.' || relname AS table_name,
pg_size_pretty(pg_relation_size(relid)) AS table_size,
n_live_tup,
n_dead_tup,
round(100.0 * n_dead_tup /
NULLIF(n_live_tup + n_dead_tup, 0), 2) AS dead_pct,
last_autovacuum::date AS last_vac,
last_autoanalyze::date AS last_ana
FROM pg_stat_user_tables
WHERE n_live_tup + n_dead_tup > 1000
ORDER BY n_dead_tup DESC
LIMIT 15;" "$DB"

log " Top 10 tables by total size:"
run_sql_verbose "
SELECT schemaname || '.' || relname AS table_name,
pg_size_pretty(pg_total_relation_size(relid)) AS total_size,
pg_size_pretty(pg_relation_size(relid)) AS table_only,
pg_size_pretty(
pg_total_relation_size(relid) - pg_relation_size(relid)
) AS indexes_toast
FROM pg_stat_user_tables
ORDER BY pg_total_relation_size(relid) DESC
LIMIT 10;" "$DB"

log " Unused indexes (0 scans since last stats reset):"
run_sql_verbose "
SELECT schemaname || '.' || relname AS table,
indexrelname AS index,
pg_size_pretty(pg_relation_size(indexrelid)) AS index_size,
idx_scan AS scans
FROM pg_stat_user_indexes
WHERE idx_scan = 0
AND indexrelname NOT LIKE '%_pkey'
AND pg_relation_size(indexrelid) > 1048576
```

```
ORDER BY pg_relation_size(indexrelid) DESC
LIMIT 15;" "$DB"

done
```

PART IX — REPLICATION AND PATRONI HEALTH CHECKS

`replication-health.sh`

Streaming replication lag, Patroni status, and slot monitoring

```
#!/bin/bash
# replication-health.sh — Replication and Patroni health checks
source /scripts/pg-dba-lib.sh

LAG_THRESHOLD="${LAG_THRESHOLD:-52428800}" # 50MB in bytes
SLOT_LAG_THRESHOLD="${SLOT_LAG_THRESHOLD:-104857600}" # 100MB

sep
log "=== Replication Health Check ==="
sep

# ■■ Patroni Cluster Status ■■
log "--- Patroni Cluster Members ---"
run_patronictl list "$CLUSTER" | tee -a "$LOG_FILE"

# ■■ Streaming Replication Status ■■
log "\n--- Streaming Replication ---"
run_sql_verbose "
SELECT client_addr,
application_name,
state,
sync_state,
pg_size_pretty(pg_wal_lsn_diff(sent_lsn, replay_lsn)) AS replay_lag,
pg_size_pretty(pg_wal_lsn_diff(sent_lsn, write_lsn)) AS write_lag,
pg_size_pretty(pg_wal_lsn_diff(sent_lsn, flush_lsn)) AS flush_lag
FROM pg_stat_replication;"

# ■■ Check for excessive lag ■■
LAGGING=$(run_sql "
SELECT application_name || ':' ||
pg_wal_lsn_diff(sent_lsn, replay_lsn)::bigint
FROM pg_stat_replication
WHERE pg_wal_lsn_diff(sent_lsn, replay_lsn) > $LAG_THRESHOLD;")

if [[ -n "$LAGGING" ]]; then
warn "Replicas with excessive lag: $LAGGING"
else
log "All replica lag within threshold"
fi

# ■■ Replication Slots ■■
log "\n--- Replication Slots ---"
run_sql_verbose "
SELECT slot_name, slot_type, active,
pg_size_pretty(
pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)
) AS retained_wal
FROM pg_replication_slots
```

```
ORDER BY pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn) DESC;"

# Alert on inactive slots with large WAL retention
BLOATED_SLOTS=$(run_sql "
SELECT slot_name
FROM pg_replication_slots
WHERE NOT active
AND pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn) > $SLOT_LAG_THRESHOLD;")

if [[ -n "$BLOATED_SLOTS" ]]; then
warn "INACTIVE slots retaining excessive WAL: $BLOATED_SLOTS"
warn "Consider dropping: SELECT pg_drop_replication_slot('<name>');"
fi

# ■■ WAL Generation Rate ■■
log "\n--- WAL Generation (last hour) ---"
run_sql_verbose "
SELECT pg_size_pretty(pg_wal_lsn_diff(
pg_current_wal_lsn(),
'0/0'::pg_lsn
)) AS total_wal_generated,
pg_size_pretty(
sum(size)
) AS current_wal_dir_size
FROM pg_ls_waldir();"

log "=== Replication Health Check Complete ==="
```

PART X — SECURITY AUDIT AND COMPLIANCE

security-audit.sh

Privilege audit, superuser check, password policy enforcement

```
#!/bin/bash
# security-audit.sh — Security audit and compliance checks
source /scripts/pg-dba-lib.sh

sep
log "=== Security Audit Report ==="
sep

# ■■ Superuser Accounts ■■
log "--- Superuser Accounts ---"
SUPERS=$(run_sql "
SELECT rolname FROM pg_roles
WHERE rolsuper = true
ORDER BY rolname;")
log "Superusers: $SUPERS"
SU_COUNT=$(echo "$SUPERS" | grep -c . || true)
if [[ "$SU_COUNT" -gt 2 ]]; then
warn "More than 2 superuser accounts detected ($SU_COUNT)"
fi

# ■■ Roles with CREATEDB / CREATEROLE ■■
log "--- Privileged Roles ---"
run_sql_verbose "
SELECT rolname,
CASE WHEN rolsuper THEN 'SUPERUSER ' ELSE '' END ||
CASE WHEN rolcreatedb THEN 'CREATEDB ' ELSE '' END ||
CASE WHEN rolcreaterole THEN 'CREATEROLE ' ELSE '' END ||
CASE WHEN rolreplication THEN 'REPLICATION ' ELSE '' END
AS privileges
FROM pg_roles
WHERE (rolsuper OR rolcreatedb OR rolcreaterole OR rolreplication)
AND rolname NOT LIKE 'pg_%'
ORDER BY rolname;"

# ■■ Roles without password expiration ■■
log "--- Roles Without Password Expiry ---"
run_sql_verbose "
SELECT rolname, rolvaliduntil
FROM pg_roles
WHERE rolcanlogin = true
AND (rolvaliduntil IS NULL OR rolvaliduntil > now() + interval '365 days')
AND rolname NOT LIKE 'pg_%'
AND rolname != 'postgres'
ORDER BY rolname;"

# ■■ pg_hba.conf review ■■
log "--- Authentication Configuration (pg_hba) ---"
run_sql_verbose "
```

```
SELECT line_number, type, database, user_name, address, auth_method
FROM pg_hba_file_rules
WHERE error IS NULL
ORDER BY line_number;"

# Flag any 'trust' or 'password' (plaintext) methods
INSECURE=$(run_sql "
SELECT count(*) FROM pg_hba_file_rules
WHERE auth_method IN ('trust', 'password');")
if [[ "$INSECURE" -gt 0 ]]; then
warn "$INSECURE insecure auth rules found (trust/password)"
fi

# ■■ SSL Status ■■
log "--- SSL Configuration ---"
SSL_ON=$(run_sql "SHOW ssl;")
log "SSL enabled: $SSL_ON"
run_sql_verbose "
SELECT count(*) AS total_conns,
count(*) FILTER (WHERE ssl) AS ssl_conns,
count(*) FILTER (WHERE NOT ssl) AS non_ssl_conns
FROM pg_stat_ssl
JOIN pg_stat_activity USING (pid);"

# ■■ Databases with public CONNECT ■■
log "--- Default Public Access ---"
run_sql_verbose "
SELECT datname,
has_database_privilege('public', datname, 'CONNECT') AS public_connect
FROM pg_database
WHERE datistemplate = false;"

log "=== Security Audit Complete ==="
```

PART XI — DAILY HEALTH CHECK REPORT GENERATOR

daily-health-report.sh

Comprehensive daily health report combining all checks

```
#!/bin/bash
# daily-health-report.sh — Comprehensive daily health check
# This is the "master" script that calls all other scripts
source /scripts/pg-dba-lib.sh

REPORT_FILE="$LOG_DIR/health-report-$(date +%Y%m%d).txt"

sep
log "========================================"
log " DAILY HEALTH CHECK REPORT"
log " Cluster: $CLUSTER"
log " Date: $(date +'%Y-%m-%d %H:%M:%S')"
log "========================================"
sep

# ■■ 1. Cluster Overview ■■
log "\n[1/8] CLUSTER OVERVIEW"
run_sql_verbose "
SELECT version();
SELECT pg_postmaster_start_time() AS uptime_since;
SELECT count(*) AS total_connections FROM pg_stat_activity;
SELECT setting AS max_connections FROM pg_settings
WHERE name = 'max_connections';"

CONN_COUNT=$(run_sql "SELECT count(*) FROM pg_stat_activity;")
MAX_CONN=$(run_sql "SELECT setting FROM pg_settings WHERE name='max_connections';")
CONN_PCT=$(( CONN_COUNT * 100 / MAX_CONN ))
if [[ "$CONN_PCT" -gt 80 ]]; then
warn "Connection usage at ${CONN_PCT}% ($CONN_COUNT / $MAX_CONN)"
else
log "Connection usage: ${CONN_PCT}% ($CONN_COUNT / $MAX_CONN)"
fi

# ■■ 2. Database Sizes ■■
log "\n[2/8] DATABASE SIZES"
run_sql_verbose "
SELECT datname,
pg_size_pretty(pg_database_size(datname)) AS size
FROM pg_database
WHERE datistemplate = false
ORDER BY pg_database_size(datname) DESC;"

# ■■ 3. Replication Status ■■
log "\n[3/8] REPLICATION STATUS"
run_patronictl list "$CLUSTER" | tee -a "$LOG_FILE"
run_sql_verbose "
SELECT application_name, state, sync_state,
pg_size_pretty(pg_wal_lsn_diff(sent_lsn, replay_lsn)) AS lag
```

```
FROM pg_stat_replication;"

# ■■ 4. Backup Status ■■
log "\n[4/8] BACKUP STATUS"
kubectl exec -n "$NAMESPACE" "$PRIMARY" -c "$CONTAINER" -- \
pgbackrest info --output=text 2>&1 | tee -a "$LOG_FILE"

WAL_LAG=$(run_sql "
SELECT EXTRACT(EPOCH FROM (now()-last_archived_time))::int
FROM pg_stat_archiver;")
if [[ "$WAL_LAG" -gt 300 ]]; then
warn "WAL archive lag: ${WAL_LAG}s"
else
log "WAL archive lag: ${WAL_LAG}s (OK)"
fi

# ■■ 5. Table Bloat Summary ■■
log "\n[5/8] TOP BLOATED TABLES"
for DB in $(list_databases); do
run_sql_verbose "
SELECT schemaname||'.'||relname AS tbl,
n_dead_tup,
round(100.0*n_dead_tup/NULLIF(n_live_tup+n_dead_tup,0),1) AS dead_pct
FROM pg_stat_user_tables
WHERE n_dead_tup > 10000
ORDER BY n_dead_tup DESC
LIMIT 5;" "$DB"
done

# ■■ 6. Slow Query Summary ■■
log "\n[6/8] TOP 5 SLOW QUERIES"
run_sql_verbose "
SELECT queryid,
round(mean_exec_time::numeric, 2) AS avg_ms,
calls,
LEFT(query, 50) AS query
FROM pg_stat_statements
WHERE userid != 10
ORDER BY mean_exec_time DESC
LIMIT 5;"

# ■■ 7. Cache Hit Ratio ■■
log "\n[7/8] CACHE HIT RATIOS"
run_sql_verbose "
SELECT datname,
round(100.0 * blks_hit / NULLIF(blks_hit+blks_read,0), 2) AS hit_pct
FROM pg_stat_database
WHERE datname NOT LIKE 'template%';"

for DB in $(list_databases); do
HIT=$(run_sql "SELECT round(100.0*blks_hit/NULLIF(blks_hit+blks_read,0),2)
FROM pg_stat_database WHERE datname='$DB';")
if [[ -n "$HIT" ]] && (( $(echo "$HIT < 95" | bc -l 2>/dev/null || echo 0) )); then
warn "Cache hit ratio below 95% for $DB: ${HIT}%"
fi
```

```
done

# ■■ 8. Disk Usage ■■
log "\n[8/8] PG DATA DIRECTORY USAGE"
kubectl exec -n "$NAMESPACE" "$PRIMARY" -c "$CONTAINER" -- \
df -h /pgdata 2>&1 | tee -a "$LOG_FILE"

sep
log "Health report saved to: $LOG_FILE"
log "Alerts saved to: $ALERT_LOG"
```

PART XII — KUBERNETES DEPLOYMENT: CRONJOBS, RBAC, CONFIGMAPS

Deploying All Scripts into the Kubernetes Cluster

Step 1: Create the ConfigMap from All Scripts

```
# Package all scripts into a single ConfigMap
kubectl create configmap dba-scripts \
--from-file=pg-dba-lib.sh=./scripts/pg-dba-lib.sh \
--from-file=manage-users.sh=./scripts/manage-users.sh \
--from-file=manage-databases.sh=./scripts/manage-databases.sh \
--from-file=manage-schemas.sh=./scripts/manage-schemas.sh \
--from-file=backup-ops.sh=./scripts/backup-ops.sh \
--from-file=daily-maintenance.sh=./scripts/daily-maintenance.sh \
--from-file=performance-monitor.sh=./scripts/performance-monitor.sh \
--from-file=bloat-report.sh=./scripts/bloat-report.sh \
--from-file=replication-health.sh=./scripts/replication-health.sh \
--from-file=security-audit.sh=./scripts/security-audit.sh \
--from-file=daily-health-report.sh=./scripts/daily-health-report.sh \
-n pg-production \
--dry-run=client -o yaml | kubectl apply -f -
```

Step 2: RBAC — ServiceAccount, Role, and RoleBinding

```
# dba-automation-rbac.yaml
apiVersion: v1
kind: ServiceAccount
metadata:
name: dba-automation
namespace: pg-production
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
name: dba-automation-role
namespace: pg-production
rules:
- apiGroups: [""]
resources: ["pods"]
verbs: ["get", "list"]
- apiGroups: [""]
resources: ["pods/exec"]
verbs: ["create"]
- apiGroups: [""]
resources: ["secrets"]
verbs: ["get", "create", "update", "patch", "delete"]
- apiGroups: ["pgv2.percona.com"]
resources: ["perconapgbackups", "perconapgclusters"]
```

```
verbs: ["get", "list", "create", "delete"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
name: dba-automation-binding
namespace: pg-production
subjects:
- kind: ServiceAccount
name: dba-automation
roleRef:
kind: Role
name: dba-automation-role
apiGroup: rbac.authorization.k8s.io
```

Step 3: CronJob Definitions

Each CronJob uses the same pattern: mount the scripts ConfigMap, use the dba-automation ServiceAccount, and run the specific script with appropriate arguments.

```
# dba-cronjobs.yaml — All scheduled DBA automation
apiVersion: batch/v1
kind: CronJob
metadata:
name: dba-daily-health
namespace: pg-production
spec:
schedule: "0 6 * * *" # 6:00 AM daily
successfulJobsHistoryLimit: 7
failedJobsHistoryLimit: 3
concurrencyPolicy: Forbid
jobTemplate:
spec:
template:
spec:
serviceAccountName: dba-automation
containers:
- name: dba
image: bitnami/kubectl:latest
command: ["/bin/bash", "/scripts/daily-health-report.sh"]
env:
- name: PG_NAMESPACE
value: "pg-production"
- name: PG_CLUSTER
value: "my-cluster"
volumeMounts:
- name: scripts
mountPath: /scripts
volumes:
- name: scripts
configMap:
name: dba-scripts
defaultMode: 0755
```

```
restartPolicy: OnFailure
---
apiVersion: batch/v1
kind: CronJob
metadata:
name: dba-daily-maintenance
namespace: pg-production
spec:
schedule: "30 2 * * *" # 2:30 AM daily
successfulJobsHistoryLimit: 7
failedJobsHistoryLimit: 3
concurrencyPolicy: Forbid
jobTemplate:
spec:
template:
spec:
serviceAccountName: dba-automation
containers:
- name: dba
image: bitnami/kubectl:latest
command: ["/bin/bash", "/scripts/daily-maintenance.sh"]
env:
- name: PG_NAMESPACE
value: "pg-production"
- name: PG_CLUSTER
value: "my-cluster"
- name: BLOAT_THRESHOLD
value: "20"
volumeMounts:
- name: scripts
mountPath: /scripts
volumes:
- name: scripts
configMap:
name: dba-scripts
defaultMode: 0755
restartPolicy: OnFailure
---
apiVersion: batch/v1
kind: CronJob
metadata:
name: dba-query-killer
namespace: pg-production
spec:
schedule: "*/10 * * * *" # Every 10 minutes
successfulJobsHistoryLimit: 3
failedJobsHistoryLimit: 3
concurrencyPolicy: Forbid
jobTemplate:
spec:
template:
spec:
serviceAccountName: dba-automation
containers:
- name: dba
```

```
image: bitnami/kubectl:latest
command: ["/bin/bash", "/scripts/performance-monitor.sh", "kill-long"]
env:
- name: PG_NAMESPACE
value: "pg-production"
- name: PG_CLUSTER
value: "my-cluster"
- name: KILL_THRESHOLD
value: "1800"
volumeMounts:
- name: scripts
mountPath: /scripts
volumes:
- name: scripts
configMap:
name: dba-scripts
defaultMode: 0755
restartPolicy: OnFailure
---
apiVersion: batch/v1
kind: CronJob
metadata:
name: dba-weekly-security-audit
namespace: pg-production
spec:
schedule: "0 7 * * 1" # Monday 7:00 AM
successfulJobsHistoryLimit: 4
failedJobsHistoryLimit: 2
jobTemplate:
spec:
template:
spec:
serviceAccountName: dba-automation
containers:
- name: dba
image: bitnami/kubectl:latest
command: ["/bin/bash", "/scripts/security-audit.sh"]
env:
- name: PG_NAMESPACE
value: "pg-production"
- name: PG_CLUSTER
value: "my-cluster"
volumeMounts:
- name: scripts
mountPath: /scripts
volumes:
- name: scripts
configMap:
name: dba-scripts
defaultMode: 0755
restartPolicy: OnFailure
---
apiVersion: batch/v1
kind: CronJob
metadata:
```

```
name: dba-weekly-bloat-report
namespace: pg-production
spec:
schedule: "0 4 * * 3" # Wednesday 4:00 AM
successfulJobsHistoryLimit: 4
failedJobsHistoryLimit: 2
jobTemplate:
spec:
template:
spec:
serviceAccountName: dba-automation
containers:
- name: dba
image: bitnami/kubectl:latest
command: ["/bin/bash", "/scripts/bloat-report.sh"]
env:
- name: PG_NAMESPACE
value: "pg-production"
- name: PG_CLUSTER
value: "my-cluster"
volumeMounts:
- name: scripts
mountPath: /scripts
volumes:
- name: scripts
configMap:
name: dba-scripts
defaultMode: 0755
restartPolicy: OnFailure
```

Apply everything:

```
kubectl apply -f dba-automation-rbac.yaml
kubectl apply -f dba-cronjobs.yaml

# Verify CronJobs are scheduled
kubectl get cronjobs -n pg-production

# Manually trigger a job for testing
kubectl create job --from=cronjob/dba-daily-health \
dba-test-run -n pg-production

# Watch logs
kubectl logs job/dba-test-run -n pg-production -f
```

PART XIII — SCRIPT INVENTORY AND SCHEDULING MATRIX

Script	CronJob Name	Schedule	Purpose
`daily-health-report.sh`	dba-daily-health	`0 6 * * *`	Full cluster health check
`daily-maintenance.sh`	dba-daily-maintenance	`30 2 * * *`	VACUUM, ANALYZE, REINDEX
`performance-monitor.sh kill-long`	dba-query-killer	`*/10 * * * *`	Kill queries > 30 min
`performance-monitor.sh kill-idle-txn`	dba-idle-txn-killer	`*/15 * * * *`	Kill idle-in-txn > 10 min
`replication-health.sh`	dba-repl-check	`*/30 * * * *`	Replication lag monitoring
`security-audit.sh`	dba-weekly-security	`0 7 * * 1`	Weekly security audit
`bloat-report.sh`	dba-weekly-bloat	`0 4 * * 3`	Weekly bloat analysis
`backup-ops.sh verify-pitr`	dba-pitr-check	`0 */6 * * *`	PITR readiness every 6h
`backup-ops.sh cleanup-old`	dba-backup-cleanup	`0 5 * * 0`	Clean old backup CRs
`manage-schemas.sh update-extensions`	dba-ext-update	`0 3 1 * *`	Monthly extension updates
`manage-users.sh audit`	dba-user-audit	`0 7 1 * *`	Monthly user audit

Ad-Hoc Scripts (Run Manually as Needed)

Script	Action	Usage Example
`manage-users.sh`	create	`manage-users.sh create app_user mydb`
`manage-users.sh`	delete	`manage-users.sh delete old_user`
`manage-users.sh`	rotate-password	`manage-users.sh rotate-password app_user`
`manage-databases.sh`	create	`manage-databases.sh create analytics etl_user`
`manage-databases.sh`	drop	`manage-databases.sh drop old_staging`
`manage-databases.sh`	clone	`manage-databases.sh clone prod_db test_copy`
`manage-schemas.sh`	create-schema	`manage-schemas.sh create-schema mydb reporting`
`backup-ops.sh`	trigger	`backup-ops.sh trigger repo1 full`
`performance-monitor.sh`	report	`performance-monitor.sh report`
`performance-monitor.sh`	reset-stats	`performance-monitor.sh reset-stats`

Running Ad-Hoc Scripts from a Bastion

```
# From a bastion VM with kubectl configured:
export PG_NAMESPACE="pg-production"
export PG_CLUSTER="my-cluster"

# Create a user
./scripts/manage-users.sh create reporting_user analytics_db

# Trigger an on-demand backup
./scripts/backup-ops.sh trigger repo1 full

# Generate a full performance report
./scripts/performance-monitor.sh report

# Or run ad-hoc via a temporary K8s job:
kubectl run dba-adhoc --rm -it \
--image=bitnami/kubectl:latest \
--serviceaccount=dba-automation \
-n pg-production \
--overrides='{
"spec": {
"containers": [{
"name": "dba-adhoc",
"image": "bitnami/kubectl:latest",
"command": ["/bin/bash"],
"stdin": true, "tty": true,
"volumeMounts": [{"name":"scripts","mountPath":"/scripts"}],
"env": [
{"name":"PG_NAMESPACE","value":"pg-production"},
{"name":"PG_CLUSTER","value":"my-cluster"}
]
}],
"volumes": [{"name":"scripts",
"configMap":{"name":"dba-scripts","defaultMode":493}}]
}
}' -- /bin/bash
```

Key Takeaway: This runbook provides 11 production-ready scripts covering the full spectrum of PostgreSQL DBA automation on Kubernetes. All scripts source a common library (`pg-dba-lib.sh`) for consistent logging, pod discovery, and error handling. Deploy them as a ConfigMap with Kubernetes CronJobs for automated execution, or run them ad-hoc from a bastion VM. The scheduling matrix in Part XIII provides recommended cron schedules calibrated for production workloads.